result. `$guitars[1]` has the value `'Stratocaster'`, so that's what the two functions return. (`$guitars[2]` would also return `true` but neither function gets that far.) Two Strats *and* a Suhr feels a little extra to me but who am I to judge?

The next two functions are slightly different. `array_any()` returns `true` if just one callback iteration returns `true`, and `array_all()` returns `true` if every callback iteration returns `true`. Here they are in action:

```
$artists = [
    'Jimi Hendrix' => 'Stratocaster',
    'Eric Clapton' => 'Les Paul',
    'Dave Gilmour' => 'Stratocaster',
];

$anyStrats = array_any(
    $artists,
    fn ($val): bool => $val === 'Stratocaster'
);
$allStrats = array_all(
    $artists,
    fn ($val): bool => $val === 'Stratocaster'
);

var_dump($anyStrats);
var_dump($allStrats);

Output:
bool(true)
bool(false)
```

The example above demonstrates some fairly clear logic: only some of the artists played Strats (OK, I know Clapton changed to Fenders, but he was always best with a Gibson in his hands). One tiny difference between `array_any()` and `array_all()` is that the former only iterates to the first `true` result, so the callback isn't firing unnecessarily, and `array_all()`, of course, has to iterate the entire array.

The two examples above are relatively trivial and simply demonstrate the use of these new functions. Remember that you can process keys and values together and implement any logic you like in the callbacks, which opens up a lot of possibilities.